

# Predictive Maintenance Project

Interview Prep & Key Facts — Random Forest Classifier on the AI4I 2020 Dataset

## Project Summary

This project trains a Random Forest Classifier to predict machine failure from five sensor readings — air temperature, process temperature, rotational speed, torque, and tool wear — using the AI4I 2020 Predictive Maintenance dataset. It also includes a Streamlit web app that takes live sensor readings and returns a failure prediction from the trained model.

## Key Facts to Remember

### Dataset

| Fact             | Value                                                                                              |
|------------------|----------------------------------------------------------------------------------------------------|
| Dataset          | AI4I 2020 Predictive Maintenance (UCI / Kaggle)                                                    |
| Rows / columns   | 10,000 rows, 14 columns                                                                            |
| Target column    | Machine failure (0 = no failure, 1 = failure)                                                      |
| Failure rate     | 3.39% — 339 of 10,000 records                                                                      |
| Features used    | Air temperature [K], Process temperature [K], Rotational speed [rpm], Torque [Nm], Tool wear [min] |
| Columns excluded | UDI, Product ID, Type (not requested); TWF, HDF, PWF, OSF, RNF (data leakage — see Q4)             |

### Model configuration

| Setting            | Value                                 |
|--------------------|---------------------------------------|
| Algorithm          | RandomForestClassifier (scikit-learn) |
| Number of trees    | 300                                   |
| Class weighting    | balanced                              |
| Train / test split | 80% / 20%, stratified by target       |
| Random seed        | 42                                    |

### Test-set results (2,000 held-out records)

| Metric           | Value                           |
|------------------|---------------------------------|
| Accuracy         | 98.4%                           |
| Precision        | 89.1%                           |
| Recall           | 60.3%                           |
| F1 score         | 71.9%                           |
| ROC-AUC          | 96.2%                           |
| Confusion matrix | TP 41 / FN 27 / TN 1,927 / FP 5 |

## Feature importance ranking

| Rank | Feature                 | Importance |
|------|-------------------------|------------|
| 1    | Torque [Nm]             | 32.3%      |
| 2    | Rotational speed [rpm]  | 30.9%      |
| 3    | Tool wear [min]         | 21.7%      |
| 4    | Air temperature [K]     | 9.2%       |
| 5    | Process temperature [K] | 5.9%       |

# Likely Interview Questions & Answers

## Project overview and motivation

### Q1. What problem does this project solve?

It predicts whether a machine will fail based on live sensor readings, so maintenance can happen before an unplanned breakdown rather than after — the core idea behind predictive maintenance.

### Q2. What is the difference between reactive, preventive, and predictive maintenance?

Reactive maintenance fixes equipment after it breaks. Preventive maintenance follows a fixed schedule regardless of actual condition. Predictive maintenance uses real-time condition data, like this model does, to anticipate failure before it happens.

### Q3. Why is the AI4I 2020 dataset a reasonable stand-in for a real production line?

It is synthetic but modeled on real milling machine behavior, with simultaneous numeric sensor readings plus an overall failure label and failure-mode subtypes — useful both for classification practice and for illustrating data leakage.

## Data and preprocessing

### Q4. Why did you exclude TWF, HDF, PWF, OSF, and RNF from the features?

They record which failure mode occurred after the fact, so they would not be known before a real failure happens. Including them leaks the answer into the model and produces misleadingly perfect results.

### Q5. Why use both Air temperature and Process temperature instead of one temperature value?

The dataset provides them as two separate sensor readings — ambient air versus the process itself, which typically runs hotter. Since they capture different physical points, both were kept rather than assuming one makes the other redundant.

### Q6. How did you handle the class imbalance?

Used a stratified train/test split so both sets kept the same 3.4% failure ratio, and set `class_weight` to `balanced` so the rare failure class is penalized more heavily during training instead of being effectively ignored.

### Q7. Why not just report accuracy?

With only 3.4% of records being failures, always predicting no failure would already score 96.6% accuracy while catching zero real failures. Precision, recall, F1, and ROC-AUC give a far more honest picture of performance on the class that matters.

## Model choice

### Q8. Why Random Forest instead of logistic regression, SVM, or a neural network?

It handles nonlinear relationships and feature interactions without needing feature scaling, is robust to outliers, performs well on structured sensor data with minimal tuning, and gives built-in feature importances — directly useful for this project's engineering question.

**Q9. What are Random Forest's limitations here?**

It can still struggle with recall on a rare class under a hard imbalance, as seen with 60.3% recall; it is less interpretable at the level of any single tree; and it can overfit if given too many, too-deep trees relative to the data available.

**Q10. What does class\_weight equals balanced actually change?**

It re-weights each class inversely to its frequency in the training data, so the rare failure class contributes proportionally more to how each tree's splits are chosen.

## Evaluation and results

**Q11. Walk through your confusion matrix.**

Of 2,000 test records: 1,927 healthy machines were correctly identified, 5 healthy machines were incorrectly flagged as failing, 27 real failures were missed, and 41 real failures were correctly caught.

**Q12. Your recall is only 60%. Is that a problem?**

It means roughly 4 in 10 real failures in the test set were missed, which matters a great deal depending on the cost of an undetected failure. It is an honest limitation worth naming directly rather than hiding behind the 98.4% accuracy figure.

**Q13. How would you improve recall?**

Lower the classification threshold to trade some precision for recall, gather more failure examples, try resampling techniques such as SMOTE, tune hyperparameters with recall as the target metric, or test gradient-boosted trees such as XGBoost.

**Q14. What does a ROC-AUC of 0.96 tell you that accuracy doesn't?**

It measures how well the model ranks failures above non-failures across every possible decision threshold, not just the default 0.5 cutoff. A 0.96 means the two classes are separated very well even though the chosen threshold gives an imperfect recall.

## Feature importance and engineering interpretation

**Q15. Which factor is most predictive of failure, and does that match mechanical intuition?**

Torque, closely followed by rotational speed — together about 63% of the model's decisions. That matches mechanical intuition: excess torque and speed raise mechanical stress and heat at the tool-workpiece interface, classic precursors to failure in machining.

**Q16. Why does tool wear matter less than torque and speed?**

It is still meaningful at 21.7%, but wear accumulates gradually, and some of its effect likely shows up indirectly through torque and speed patterns as a tool wears down.

**Q17. Is feature importance the same as proving causation?**

No. Feature importances reflect how useful a feature was for reducing impurity across the forest's splits, which correlates with predictive power but is not a validated causal claim on its own.

## Web app and deployment

**Q18. Why Streamlit instead of a plain HTML and JavaScript page?**

Streamlit wraps a trained scikit-learn model in an interactive form with very little code, loading the saved model file directly without reimplementing the trained logic in another language or standing up a separate backend API.

**Q19. The brief only mentioned temperature and speed, but the app asks for five inputs. Why?**

The model was trained on all five features together, so feeding it only two and guessing at the rest would silently bias every prediction. Asking for all five keeps predictions faithful to how the model actually learned.

**Q20. How would this move from a demo to real factory use?**

It would need a continuous sensor data pipeline instead of manual form entry, monitoring for data or concept drift, a proper backend and API layer, logging and alerting integration, and a periodic retraining schedule as real failure data accumulates.

## Tougher Follow-Up Questions

### **Q21. How do you know this model isn't overfitting?**

All reported metrics come from a held-out test set the model never trained on, which is the main defense against overfitting. A stronger check would add k-fold cross-validation across several splits to confirm the results are stable, not a fluke of one split.

### **Q22. What would change if you used SMOTE instead of class weighting?**

SMOTE creates new synthetic minority-class examples by interpolating between real ones, which can help but risks generating sensor readings that don't correspond to anything physically real. Class weighting avoids that by only reweighting existing real samples.

### **Q23. If you had to keep only two features, which would you pick, and what would you lose?**

Torque and rotational speed, since together they carry about 63% of the model's importance. You would lose the tool wear signal (21.7%) and both temperature signals (about 15% combined), likely reducing recall further.

### **Q24. What would you change if this were a real production deployment rather than a class project?**

More historical failure data across a longer time horizon, cross-validation instead of a single split, threshold tuning against the real cost of a missed failure versus a false alarm, and a monitoring plan to catch when the model's assumptions stop matching new production data.